



Apéndice: Cartilla de Assembler Z-80

1. Características del Procesador Z-80

RDM de 2 bytes (0000h al FFFFh)

RBM de 1 bytes (8 bits en cada Transferencia de Memoria)

El almacenamiento se realiza usando Little-Endian

1.1. Registros

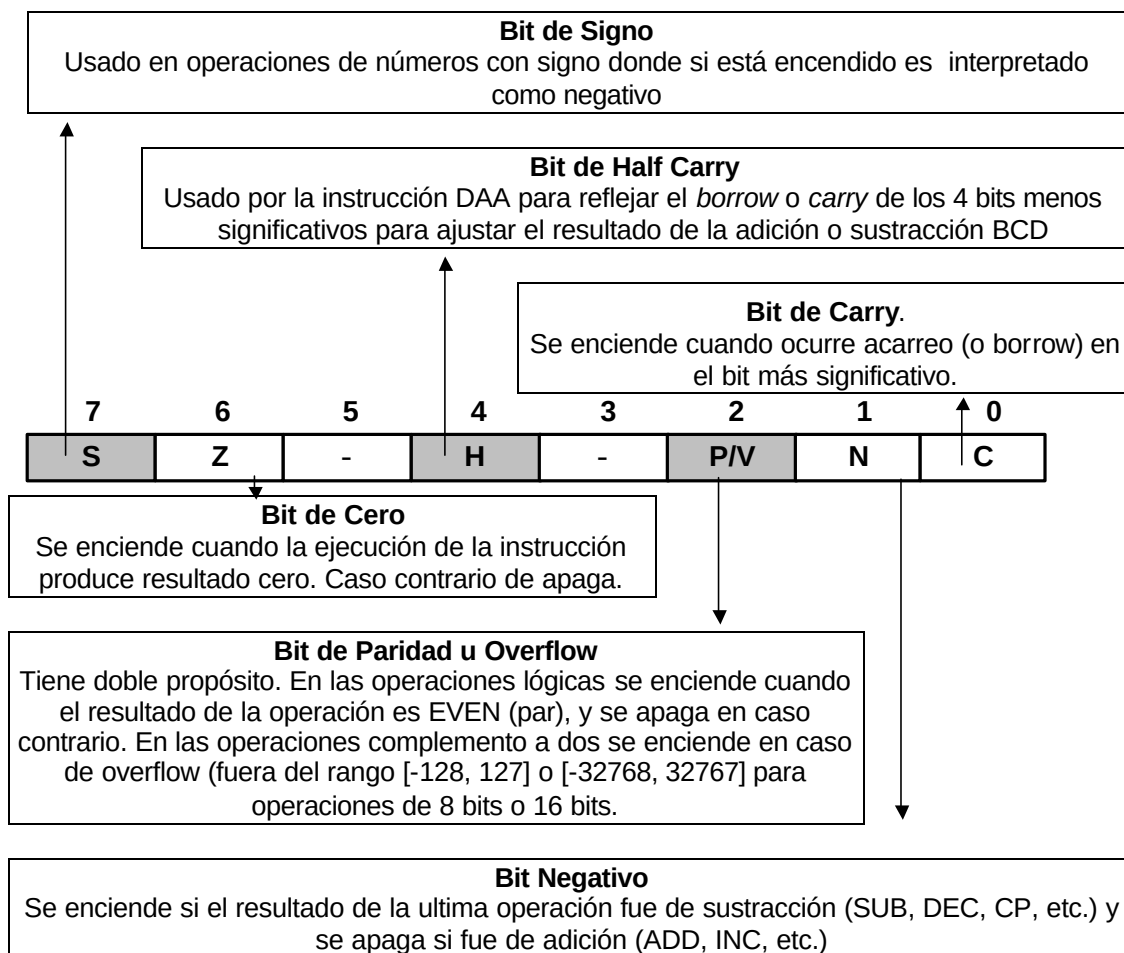
Existen **tres grupos** de registros: **grupo GR**, **grupo GR'** y **grupo de Registros Especiales**. La diferencia entre los del grupo GR' con los del GR es que sus contenido **no son directamente accesibles**, sólo pueden ser utilizados para intercambiarse con los del grupo GR a alta velocidad.

Registros Especiales:

- **I:** Registro de Interrupción (8 bits). Indica el lugar del vector de Interrupción.
- **R:** Registro Contador ó Refresh (8 bits). Los 7 bits menos significativos indican la cantidad de instrucciones ejecutadas por el procesador. Es incrementado por el procesador en cada fetch.
- **IX, IY:** Registros Índices (16 bits). Se usan para manipular datos y direcciones. Cuando se usa en direcciones, el contenido de un desplazamiento especificado en la instrucción es sumado o restado del registro índice para calcular la dirección efectiva de un operando.
- **SP:** Registro Stack Pointer (16 bits). Contiene la dirección del Stack.
- **PC:** Registro Contador de Programa (16 bits). Contiene la dirección de la próxima instrucción a ejecutar. Es actualizado automáticamente después de cada fetch.

Registros del grupo GR:

- **A:** Registro Acumulador (8 bits). Se utiliza para operaciones aritméticas, lógicas e instrucciones de E/S.
- **B, C, D, E, H, L:** Registros de Uso General para manipular datos de 8 bits. Pueden ser tratados en forma individual (B, C, D, E, H, L) o aparearse en tres pares (BC, DE, HL) para manipular 16 bits.
- **F:** Registro de Flag (8 bits). Guarda los bits de estados que guardan los resultados de la ejecución de instrucciones. Se consulta para controlar el flujo de ejecución del programa y la operación de la instrucción. El significado de cada uno de sus bits se detalla en el siguiente cuadro:



1.2. Modos de Direccionamiento

- ☞ **Registro Implícito:** Ciertas instrucciones se aplican automáticamente a un registro predeterminado. Por ejemplo las instrucciones aritméticas y lógicas utilizan direccionamiento implícito, ya que todas ellas realizan operaciones con el contenido del registro acumulador A.
- ☞ **Registro Directo:** El código de operación de la instrucción contiene cuál o cuáles registros están implicados en la ejecución de la instrucción.
- ☞ **Registro Indirecto:** Alguno de los registros de uso general (16 bits: BC, DE o HL) indican la dirección donde se encuentra el operando de memoria.
- ☞ **Registro Indexado:** La dirección del operando en memoria es calculada usando el contenido de uno de los registros índice IX o IY más un desplazamiento de 8 bits con signo que se especifica en las instrucción.
- ☞ **Extendido:** La dirección del operando en memoria es especificada por dos bytes contenidos en la instrucción. La dirección se coloca entre paréntesis.
- ☞ **Inmediato:** El operando está contenido en 1 ó 2 bytes de la instrucción.
- ☞ **Relativo:** Se utiliza sólo para instrucciones de bifurcación condicional o incondicional. El desplazamiento es sumado al registro PC.
- ☞ **I/O:** Se utiliza sólo para instrucciones de E/S.



2. Directivas al Compilador (Pseudoinstrucciones)

Son acciones que debe tomar el compilador, pero **NO GENERAN CÓDIGO**.

- **Definición de Constantes Simbólicas**

Asignan un nombre simbólico a una expresión. Existen dos formas de hacerlo:

<i>nombre</i>	equ	<i>expresión</i>
---------------	------------	------------------

<i>nombre</i>	defl	<i>expresión</i>
---------------	-------------	------------------

- **Definición de Datos**

Reservan posiciones de memoria para guardar variables. Existen 5 formas de hacerlo, alguna de ellas permiten inicializar con un valor y otras sólo reservar espacio:

<i>nombre</i>	db	<i>listaDeExpresiones</i>	(cada dato en 1 byte)
---------------	-----------	---------------------------	-----------------------

<i>nombre</i>	dw	<i>listaDeExpresiones</i>	(cada dato en un doble byte)
---------------	-----------	---------------------------	------------------------------

<i>nombre</i>	defb	<i>mensajeDeCaracteres</i>	(cada caracter en 1 byte)
---------------	-------------	----------------------------	---------------------------

<i>nombre</i>	deff	<i>listaDeNrosReales</i>	(IEEE 754 de 32 bits)
---------------	-------------	--------------------------	-----------------------

<i>nombre</i>	ds	<i>cantidadBytes</i>	(sólo reserva espacio)
---------------	-----------	----------------------	------------------------

- **Control de Ensamblado**

Indica que la primera instrucción a ejecutar (no necesariamente la primera ejecutable) es la que se encuentra en la dirección indicada por rótulo:

end	<i>rótulo</i>
------------	---------------



- **Referencias Externas**

Para indicar que pueden ser usados por otros módulos (análogo a colocar el “..” haciendo el rótulo o constante simbólica pública) se usa:

public	<i>listaDeIdentificadores</i>
---------------	-------------------------------

ent	<i>listaDeIdentificadores</i>
------------	-------------------------------

Para indicar que los rótulos o constantes simbólicas están definidas en otros módulos y es el linkeditor el que debe resolverlas, se coloca:

extern	<i>listaDeIdentificadores</i>
---------------	-------------------------------

external	<i>listaDeIdentificadores</i>
-----------------	-------------------------------

- **Directivas de Segmento**

Segmento absoluto:

Aseg org	<i>direccionDeMemoria</i>
---------------------	---------------------------

Segmento Reubicable:

Cseg
.....
<i>zonaDeCodigo</i>
.....

Dseg
.....
<i>zonaDeDatos</i>
.....



3. Set de Instrucciones Detalladas



Explica el modo de direccionamiento de los operandos que intervienen en la instrucción y el funcionamiento de la misma.



Indica el formato de la instrucción (forma de uso para incluirla en un código fuente)



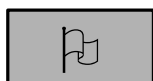
Indica la cantidad de ciclos de memoria utilizados en la ejecución de la instrucción



Indica la cantidad de ciclos de CPU utilizados en la ejecución de la instrucción



Muestra el código de operación (código de máquina) asociado a la instrucción (y por lo tanto la longitud de la instrucción)



Muestra el estado de los bits del registro F después de la ejecución de la instrucción, según la siguiente convención:

- ☒ si es alterado por la operación
- ☒ si no es alterado por la operación
- 0** si es apagado por la operación
- 1** si es encendido por la operación
- ?** si es indefinido después de la operación

En el caso de que una instrucción setee el bit **P/V** (Paridad/Overflow), se indica con una **P** o con una **V** según esté indicando Paridad u Overflow, respectivamente.

TABLA 1	
Regist ro	Códig o
B	000
C	001
D	010
E	011
H	100
L	101
-	110
A	111

TABLA 2	
Regist ro	Códig o
BC	00
DE	01
HL	10
AF	11

TABLA 3	
Regist ro	Códig o
BC	00
DE	01
HL	10
SP	11

TABLA 4	
Regist ro	Códig o
BC	00
DE	01
IX	10
SP	11

TABLA 5	
Regist ro	Códig o
BC	00
DE	01
IY	10
SP	11

TABLA 6		
Flag	Significado	Códig o
NZ	No zero	000
Z	Zero	001
NC	No carry	010
C	Carry	011
PO	Paridad impar	100
PE	Paridad par	101
P	Signo mas	110
M	Signo menos	111



3.1. Instrucciones Aritméticas (8 y 16 bits)

Add (8 bits)

La suma de 8 bits utiliza el **modo de direccionamiento implícito**, ya que siempre deja el resultado en el registro acumulador A.

Además, el otro operando que interviene en la suma puede tener diferentes modos de direccionamiento, que se indican en la tabla. La longitud de instrucción varía según el operando en cuestión.

					S	Z	H	P V	N	C
Registro Directo $A \leftarrow A + r$	Add A, r Donde r puede ser alguno de los siguientes registros de 8 bits: B, C, D, E, H, L, A	22	4	8 ... 10000--- donde --- codifica alguno de los registros de 8 bits con la convención de la tabla 1	☒	☒	☒	☒ V	0	☒
Registro Indirecto $A \leftarrow A + (HL)$	Add A, (HL)	2	6	86	☒	☒	☒	☒ V	0	☒
Registro Indexado $A \leftarrow A + (IX + d)$	Add A, (IX + d) donde d es un desplazamiento de 8 bits	6	3	DD 86 d	☒	☒	☒	☒ V	0	☒
Registro Indexado $A \leftarrow A + (IY + d)$	Add A, (IY + d) donde d es un desplazamiento de 8 bits	6	3	FD 86 d	☒	☒	☒	☒ V	0	☒
Inmediato $A \leftarrow A + v$	Add A, v donde v es un dato de 8 bits	2	6	C6 v	☒	☒	☒	☒ V	0	☒



Add (16 bits)

La suma de 16 bits utiliza el **modo de direccionamiento implícito** y el de **registro directo**. Sólo se pueden hacer sumas entre ciertos pares de registros.

					S	Z	H	P V	N	C
Registro Directo HL ← HL + r	Add HL, r donde r puede ser alguno de los siguientes pares registros de 16 bits: BC, DE, HL, SP	5	7	... 9 00--1001 donde --- codifica alguno de los pares de registros de 16 bits con la convención de la tabla 3	☒	☒	?	☒	0	☒
Registro Directo IX ← IX + r	Add IX, r donde r puede ser alguno de los siguientes pares registros de 16 bits: BC, DE, IX, SP	6	10	DD...9 11011101 00--1001 donde --- codifica alguno de los pares de registros de 16 bits con la convención de la tabla 4	☒	☒	?	☒	0	☒
Registro Directo IY ← IY + r	Add IY, r donde r puede ser alguno de los siguientes pares registros de 16 bits: BC, DE, IY, SP	6	10	FD...9 11111101 00--1001 donde --- codifica alguno de los pares de registros de 16 bits con la convención de la tabla 5	☒	☒	?	☒	0	☒



Adb (8 bits)

La suma con carry utiliza *el modo de direccionamiento implícito*, ya que siempre deja el resultado en el registro acumulador A.

Además, el otro operando que interviene en la suma puede tener diferentes modos de direccionamiento, que se indican en la tabla. La longitud de instrucción varía según el operando en cuestión.

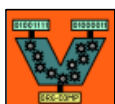
					S	Z	H	P V	N	C
Registro Directo $A \leftarrow A + r + c$	Adb A, r donde r puede ser alguno de los siguientes registros de 8 bits: B, C, D, E, H, L, A	2	4	8... 10001--- donde --- codifica alguno de los registros de 8 bits con la convención de la tabla 1	☒	☒	☒	☒ V	0	☒
Registro Indirecto $A \leftarrow A + (HL) + c$	Adb A, (HL)	6	6	8E	☒	☒	☒	☒ V	0	☒
Registro Indexado $A \leftarrow A + (IX + d) + c$	Adb A, (IX + d) donde d es un desplazamiento de 8 bits	6	14	DD 8E d	☒	☒	☒	☒ V	0	☒
Registro Indexado $A \leftarrow A + (IY + d) + c$	Adb A, (IY + d) donde d es un desplazamiento de 8 bits	6	14	FD 8E d	☒	☒	☒	☒ V	0	☒
Inmediato $A \leftarrow A + v + c$	Adb A, v donde v es un dato de 8 bits	2	6	CE v	☒	☒	☒	☒ V	0	☒



Adc (16 bits)

La suma con carry de 16 bits utiliza el **modo de direccionamiento implícito** y el **directo**. Sólo se pueden hacer sumas entre ciertos pares de registros.

					S	Z	H	P V	N	C
Registro Directo HL ← HL + r + c	Adc HL, r donde r puede ser alguno de los siguientes pares registros de 16 bits: BC, DE, HL, SP	6	10	ED...A 11101101 01--1010 donde --- codifica alguno de los pares de registros de 16 bits con la convención de la tabla 3	☒	☒	?	☒ V	0	☒

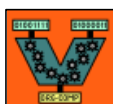


Sub (8 bits)

La diferencia de 8 bits utiliza el **modo de direccionamiento implícito**, ya que siempre deja el resultado en el registro acumulador A.

Además, el otro operando que interviene en la diferencia puede tener distintos modos de direccionamiento. La longitud de instrucción varía según el operando en cuestión.

					S	Z	H	P V	N	C
Registro Directo $A \leftarrow A - r$	Sub r donde r puede ser alguno de los siguientes registros de 8 bits: B, C, D, E, H, L, A	2	4	9... 10010--- donde --- codifica alguno de los registros de 8 bits con la convención de la tabla 1	☒	☒	☒	☒ V	1	☒
Registro Indirecto $A \leftarrow A - (HL)$	Sub (HL)	2	6	96	☒	☒	☒	☒ V	1	☒
Registro Indexado $A \leftarrow A - (IX + d)$	Sub (IX + d) donde d es un desplazamiento de 8 bits	6	14	DD 96 d	☒	☒	☒	☒ V	1	☒
Registro Indexado $A \leftarrow A - (IY + d)$	Sub (IY + d) donde d es un desplazamiento de 8 bits	6	14	FD 96 d	☒	☒	☒	☒ V	1	☒
Inmediato $A \leftarrow A - v$	Sub v donde v es un dato de 8 bits	2	6	D6 v	☒	☒	☒	☒ V	1	☒



Sbc (8 bits)

La diferencia de 8 bits con carry utiliza el **modo de direccionamiento implícito**, ya que siempre deja el resultado en el registro acumulador A.

Además, el otro operando que interviene en la diferencia puede tener diferentes modos de direccionamiento. La longitud de instrucción varía según el operando en cuestión.

					S	Z	H	P V	N	C
Registro Directo $A \leftarrow A - r - c$	Sbc A, r donde r puede ser alguno de los siguientes registros de 8 bits: B, C, D, E, H, L, A	2	4	9... 10011--- donde --- codifica alguno de los registros de 8 bits con la convención de la tabla 1	☒	☒	☒	☒ V	1	☒
Registro Indirecto $A \leftarrow A - (HL) - c$	Sbc A, (HL)	2	6	9E	☒	☒	☒	☒ V	1	☒
Registro Indexado $A \leftarrow A - (IX + d) - c$	Sbc A, (IX + d) donde d es un desplazamiento de 8 bits	6	14	DD 9E d	☒	☒	☒	☒ V	1	☒
Registro Indexado $A \leftarrow A - (IY + d) - c$	Sbc A, (IY + d) donde d es un desplazamiento de 8 bits	6	14	FD 9E d	☒	☒	☒	☒ V	1	☒
Inmediato $A \leftarrow A - v - c$	Sbc A, v donde v es un dato de 8 bits	2	6	DE v	☒	☒	☒	☒ V	1	☒



Sbc (16 bits)

La diferencia de 16 bits con carry utiliza el **modo de direccionamiento implícito** y el **directo**. Sólo puede realizarse entre determinado pares de registros.

					S	Z	H	P V	N	C
Registro Directo HL ← HL - r - c	Sbc HL, r donde r puede ser alguno de los siguientes pares de registros de 16 bits: BC, DE, HL, SP	6	10	ED...2 11101101 01--0010 donde -- codifica alguno de los pares de registros de 16 bits con la convención de la tabla 3	☒	☒	?	☒ V	1	☒



Inc (8 bits)

Esta es una operación unaria, ya que tiene un único operando el cual incrementa en 1. El operando funciona como origen y destino, ya que en el mismo se guarda el valor modificado. La longitud de instrucción varía según el operando en cuestión.

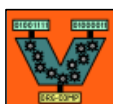
					S	Z	H	P V	N	C
Registro Directo $r \leftarrow r + 1$	Inc r donde r puede ser alguno de los siguientes registros de 8 bits: B, C, D, E, H, L, A	2	4	 00---100 donde --- codifica alguno de los registros de 8 bits con la convención de la tabla 1	☒	☒	☒	☒ V	0	☒
Registro Indirecto $(HL) \leftarrow (HL) + 1$	Inc (HL)	4	10	34	☒	☒	☒	☒ V	0	☒
Registro Indexado $(IX + d) \leftarrow (IX + d) + 1$	Inc (IX + d) donde d es un desplazamiento de 8 bits	8	18	DD 34 d	☒	☒	☒	☒ V	0	☒
Registro Indexado $(IY + d) \leftarrow (IY + d) + 1$	Inc (IY + d) donde d es un desplazamiento de 8 bits	8	18	FD 34 d	☒	☒	☒	☒ V	0	☒



Inc (16 bits)

Esta es una operación unaria, ya que tiene un único operando el cual incrementa en 1. El operando funciona como origen y destino, ya que en el mismo se guarda el valor modificado. La longitud de instrucción varía según el operando en cuestión.

										
					S	Z	H	P V	N	C
Registro Directo $r \leftarrow r + 1$	Inc r donde r puede ser alguno de los siguientes pares de registros de 16 bits: BC, DE, HL, SP	2	4	...3 00--0011 donde -- codifica alguno de los pares de registros de 16 bits con la convención de la tabla 3	☒	☒	☒	☒	☒	☒
Registro Implícito $IX \leftarrow IX + 1$	Inc IX	3	7	DD 23	☒	☒	☒	☒	☒	☒
Registro Implícito $IY \leftarrow IY + 1$	Inc IY	3	7	FD 23	☒	☒	☒	☒	☒	☒



Dec (8 bits)

Esta es una operación unaria, ya que tiene un único operando el cual se decrementa en 1. El operando funciona como origen y destino, ya que en el mismo se guarda el valor modificado. La longitud de instrucción varía según el operando en cuestión.

					S	Z	H	P V	N	C
Registro Directo $r \leftarrow r - 1$	Dec r donde r puede ser alguno de los siguientes registros de 8 bits: B, C, D, E, H, L, A	2	4	 00---101 donde --- codifica alguno de los registros de 8 bits con la convención de la tabla 1	☒	☒	☒	☒ V	1	☒
Registro Indirecto $(HL) \leftarrow (HL) - 1$	Dec (HL)	4	10	35	☒	☒	☒	☒ V	1	☒
Registro Indexado $(IX + d) \leftarrow (IX + d) - 1$	Dec (IX + d) donde d es un desplazamiento de 8 bits	8	18	DD 35 d	☒	☒	☒	☒ V	1	☒
Registro Indexado $(IY + d) \leftarrow (IY + d) - 1$	Dec (IY + d) donde d es un desplazamiento de 8 bits	8	18	FD 35 d	☒	☒	☒	☒ V	1	☒



Dec (16 bits)

Esta es una operación unaria, ya que tiene un único operando el cual se decrementa en 1. El operando funciona como origen y destino, ya que en el mismo se guarda el valor modificado. La longitud de instrucción varía según el operando en cuestión.

					S	Z	H	P V	N	C
Registro Directo $r \leftarrow r - 1$	Dec r donde r puede ser alguno de los siguientes pares de registros de 16 bits: BC, DE, HL, SP	2	4	...B 00--1011 donde -- codifica alguno de los pares de registros de 16 bits con la convención de la tabla 3	☒	☒	☒	☒	☒	☒
Registro Implícito $IX \leftarrow IX - 1$	Dec IX	3	7	DD 2B	☒	☒	☒	☒	☒	☒
Registro Implícito $IY \leftarrow IY - 1$	Dec IY	3	7	FD 2B	☒	☒	☒	☒	☒	☒



Neg

La negación sólo se puede hacer en el registro acumulador A. Representa el complemento a 2 del acumulador (aplicar NEG es lo mismo que hacer 0-A)

										
					S	Z	H	P V	N	C
Registro Implícito	Neg	2	6	ED 44	☒	☒	☒	☒	1	☒
$A \leftarrow -A$								V		



Mlt

La multiplicación aparece en assembler (es una extensión al Z80). Opera sólo con ciertos registros de 16 bits. Multiplica un byte por el otro byte dejando el resultado en el registro de 16 bits.

El operando funciona como origen y destino, ya que en el mismo se guarda el valor modificado.

Sólo sirve para multiplicar números positivos.

					S	Z	H	P V	N	C
Registro Directo $r \leftarrow r_h * r_l$	Mlt r donde r puede ser alguno de los siguientes pares de registros de 16 bits: BC, DE, HL, SP	13	17	ED...C 11101101 01--1100 donde -- codifica alguno de los pares de registros de 16 bits con la convención de la tabla 3	☒	☒	☒	☒	☒	☒



3.2. Instrucciones de Transferencia de Datos

Ld (8 bits)

					S	Z	H	P V	N	C
Registro Implícito en fuente y destino $A \leftarrow I$	Ld A, I	2	6	ED 57	☒	☒	0	☒	0	☒
Registro Implícito en fuente y destino $A \leftarrow R$	Ld A, R	2	6	ED 5F	☒	☒	0	☒	0	☒
Registro Implícito en fuente y destino $I \leftarrow A$	Ld I, A	2	6	ED 47	☒	☒	☒	☒	☒	☒
Registro Implícito en fuente y destino $R \leftarrow A$	Ld R, A	2	6	ED 4F	☒	☒	☒	☒	☒	☒
Registro Directo en fuente y en destino $r \leftarrow r'$	Ld r, r' donde r y r' son cualquier registro de la tabla 1	2	4	 01--- --- los --- primeros codifican al registro r y los otros --- codifican al registro r', según la tabla 1	☒	☒	☒	☒	☒	☒
Registro Implícito para el destino, registro Indirecto para el origen $A \leftarrow (BC)$	Ld A, (BC)	2	6	0A	☒	☒	☒	☒	☒	☒



					S	Z	H	P V	N	C
Registro Implícito para el destino, registro Indirecto para el origen $A \leftarrow (DE)$	Ld A, (DE)	2	6	1A	☒	☒	☒	☒	☒	☒
Registro Indirecto para el destino, Registro Implícito para el origen $(BC) \leftarrow A$	Ld (BC), A	6	7	02	☒	☒	☒	☒	☒	☒
Registro Indirecto para el destino, Registro Implícito para el origen $(DE) \leftarrow A$	Ld (DE), A	3	7	12	☒	☒	☒	☒	☒	☒
Registro Implícito para el destino, Extendido para el fuente $A \leftarrow (mn)$	Ld A, (mn) donde mn es un número de 16 bits	4	12	3A n m	☒	☒	☒	☒	☒	☒
Extendido para el destino, Registro Implícito para el fuente $(mn) \leftarrow A$	Ld (mn), A donde mn es un número de 16 bits	5	13	32 n m	☒	☒	☒	☒	☒	☒
Registro Directo para el destino, registro Indirecto para el fuente $r \leftarrow (HL)$	Ld r, (HL) donde r es alguno de los registros de la tabla 1	2	6	 01---110 donde --- codifica alguno de los registro de la tabla 1	☒	☒	☒	☒	☒	☒



					S	Z	H	P V	N	C
Registro Indirecto para el destino, Registro Directo para el fuente $(HL) \leftarrow r$	Ld (HL), r donde r es alguno de los registros de la tabla 1	3	7	7... 01110--- donde --- codifica alguno de los registro de tabla 1	☒	☒	☒	☒	☒	☒
Registro Directo para el destino, registro Indexado para el fuente $r \leftarrow (IX + d)$	Ld r, (IX+d) donde r es alguno de los registros de la tabla 1, y d es un desplazamiento de 8 bits	6	14	DD d 11011101 01---110 d donde --- codifica algún registro de la tabla 1	☒	☒	☒	☒	☒	☒
Registro Directo para el destino, registro Indexado para el fuente $r \leftarrow (IY + d)$	Ld r, (IY+d) donde r es alguno de los registros de la tabla 1, y d es un desplazamiento de 8 bits	6	14	FD d 11111101 01---110 d donde --- codifica algún registro de la tabla 1	☒	☒	☒	☒	☒	☒
Registro Indexado para el destino, registro Directo para el fuente $(IX + d) \leftarrow r$	Ld (IX+d), r donde r es alguno de los registros de la tabla 1, y d es un desplazamiento de 8 bits	7	15	DD 7... d 11011101 01110--- d donde --- codifica algún registro de la tabla 1	☒	☒	☒	☒	☒	☒
Registro Indexado para el destino, registro Directo para el fuente $(IY + d) \leftarrow r$	Ld (IY+d), r donde r es alguno de los registros de la tabla 1, y d es un desplazamiento de 8 bits	7	15	FD 7... d 11111101 01110--- d donde --- codifica algún registro de la tabla 1	☒	☒	☒	☒	☒	☒

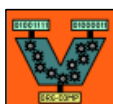


					S	Z	H	P V	N	C
Registro Directo para el destino, Inmediato para el fuente $r \leftarrow v$	Ld r, v donde r es un registro de la tabla 1 y v es un dato de 8 bits	2	6	 v 00---110 v donde --- codifica alguno de los registros de la tabla 1	☒	☒	☒	☒	☒	☒
Registro Indirecto para el destino, Inmediato para el fuente $(HL) \leftarrow v$	Ld (HL), v donde v es un dato de 8 bits	3	9	36 v	☒	☒	☒	☒	☒	☒
Registro Indexado para el destino, Inmediato para el fuente $(IX + d) \leftarrow v$	Ld (IX+d), v donde v es un dato de 8 bits	5	15	DD 36 d v	☒	☒	☒	☒	☒	☒
Registro Indexado para el destino, Inmediato para el fuente $(IY + d) \leftarrow v$	Ld (IY+d), v donde v es un dato de 8 bits	5	15	FD 36 d v	☒	☒	☒	☒	☒	☒



Ld (16 bits)

					S	Z	H	P V	N	C
Registro Implícito en fuente y destino SP ← HL	Ld SP, HL	2	4	F9	☒	☒	☒	☒	☒	☒
Registro Implícito en fuente y destino SP ← IX	Ld SP, IX	3	7	DD F9	☒	☒	☒	☒	☒	☒
Registro Implícito en fuente y destino SP ← IY	Ld SP, IY	3	7	FD F9	☒	☒	☒	☒	☒	☒
Registro Implícito para el destino, Extendido para el fuente HL ← (mn)	Ld HL, (mn) donde mn es un número de 16 bits	3	15	2A n m	☒	☒	☒	☒	☒	☒
Registro Implícito para el destino, Extendido para el fuente IX ← (mn)	Ld IX, (mn) donde mn es un número de 16 bits	6	18	DD 2A n m	☒	☒	☒	☒	☒	☒
Registro Implícito para el destino, Extendido para el fuente IY ← (mn)	Ld IY, (mn) donde mn es un número de 16 bits	6	18	FD 2A n m	☒	☒	☒	☒	☒	☒
Extendido para el destino, Registro Implícito para el fuente (mn) ← HL	Ld (mn), HL donde mn es un número de 16 bits	6	16	22 n m	☒	☒	☒	☒	☒	☒



					S	Z	H	P V	N	C
Extendido para el destino, Registro Implícito para el fuente (mn) ← IX	Ld (mn), IX donde mn es un número de 16 bits	7	19	DD 22 n m	☒	☒	☒	☒	☒	☒
Extendido para el destino, Registro Implícito para el fuente (mn) ← IY	Ld (mn), IY donde mn es un número de 16 bits	7	19	FD 22 n m	☒	☒	☒	☒	☒	☒
Registro Directo para el destino, Extendido para el fuente r ← (mn)	Ld r, (mn) donde r es alguno de los registros de la tabla 3, y mn es un dato de 16 bits	6	9	ED...B n m 11101101 01--1011 n m	☒	☒	☒	☒	☒	☒
Extendido para el destino, Registro Directo para el fuente (mn) ← r	Ld (mn), r donde r es alguno de los registros de la tabla 3 (≠HL), y mn es un dato de 16 bits	7	19	ED...3 n m 11101101 01--0011 n m	☒	☒	☒	☒	☒	☒
Registro Implícito para el destino, Inmediato para el fuente IX ← mn	Ld IX, mn donde mn es un dato de 16 bits	4	12	DD 21 n m	☒	☒	☒	☒	☒	☒
Registro Implícito para el destino, Inmediato para la fuente IY ← mn	Ld IY, mn donde mn es un dato de 16 bits	4	12	FD 21 n m	☒	☒	☒	☒	☒	☒
Registro Directo para el destino, Inmediato para el fuente r ← mn	Ld r, mn donde r es alguno de los registros de la tabla 3, y mn es un dato de 16 bits	3	9	...1 n m 00--0001 n m	☒	☒	☒	☒	☒	☒



3.3. Instrucciones Lógicas

And

La operación lógica de conjunción es de 8 bits y utiliza el **modo de direccionamiento implícito en el destino** ya que siempre deja el resultado en el registro acumulador A. El otro operando que interviene en la conjunción puede tener diferentes modos de direccionamiento.

					S	Z	H	P V	N	C
Registro Directo en fuente $A \leftarrow A \& r$	And r donde r es alguno de los siguientes registros de 8 bits: B, C, D, E, H, L, A	1	4	A ... 10100--- donde --- codifica alguno de los registros de 8 bits con la convención de la tabla 1	☒	☒	1	☒ P	0	0
Registro Indirecto $A \leftarrow A \& (HL)$	And (HL)	1	6	A6	☒	☒	1	☒ P	0	0
Registro Indexado $A \leftarrow A \& (IX + d)$	And (IX + d)	3	14	DD A6 d	☒	☒	1	☒ P	0	0
Registro Indexado $A \leftarrow A \& (IY + d)$	And (IY + d)	3	14	FD A6 d	☒	☒	1	☒ P	0	0
Inmediato $A \leftarrow A \& v$	And v	2	6	E6 v	☒	☒	1	☒ P	0	0



Or

La operación lógica de disyunción es de 8 bits y utiliza el **modo de direccionamiento implícito en el destino** ya que siempre deja el resultado en el registro acumulador A. El otro operando que interviene en la disyunción puede tener diferentes modos de direccionamiento.

					S	Z	H	P V	N	C
Registro Directo en fuente $A \leftarrow A r$	Or r donde r es alguno de los siguientes registros de 8 bits: B, C, D, E, H, L, A	1	4	B ... 10110--- donde --- codifica alguno de los registros de 8 bits con la convención de la tabla 1	☒	☒	0	☒ P	0	0
Registro Indirecto $A \leftarrow A (HL)$	Or (HL)	1	6	B6	☒	☒	0	☒ P	0	0
Registro Indexado $A \leftarrow A (IX + d)$	Or (IX + d)	3	14	DD B6 d	☒	☒	0	☒ P	0	0
Registro Indexado $A \leftarrow A (IY + d)$	Or (IY + d)	3	14	FD B6 d	☒	☒	0	☒ P	0	0
Inmediato $A \leftarrow A v$	Or v	2	6	F6 v	☒	☒	0	☒ P	0	0



Xor

La operación lógica de disyunción exclusiva es de 8 bits y utiliza el **modo de direccionamiento implícito en el destino** ya que siempre deja el resultado en el registro acumulador A. El otro operando que interviene en la disyunción exclusiva puede tener diferentes modos de direccionamiento.

										
					S	Z	H	P V	N	C
Registro Directo en fuente $A \leftarrow A \oplus r$	XOR r donde r es alguno de los siguientes registros de 8 bits: B, C, D, E, H, L, A	1	4	A ... 10101--- donde --- codifica alguno de los registros de 8 bits con la convención de la tabla 1	☒	☒	0	☒ P	0	0
Registro Indirecto $A \leftarrow A \oplus (HL)$	XOR (HL)	1	6	AE	☒	☒	0	☒ P	0	0
Registro Indexado $A \leftarrow A \oplus (IX + d)$	XOR (IX + d)	3	14	DD AE d	☒	☒	0	☒ P	0	0
Registro Indexado $A \leftarrow A \oplus (IY + d)$	XOR (IY + d)	3	14	FD AE d	☒	☒	0	☒ P	0	0
Inmediato $A \leftarrow A \oplus v$	XOR v	2	6	EE v	☒	☒	0	☒ P	0	0



Cpl

Esta instrucción realiza una negación bit a bit del registro acumulador. Es lo que se llama **complemento a 1**.

										
					S	Z	H	P V	N	C
Registro Implícito	Cpl	1	3	2F	☒	☒	1	☒	1	☒
$A \leftarrow \neg A$										



Cp

La operación de comparar es de 8 bits y utiliza el **modo de direccionamiento implícito en el destino**. El otro operando que interviene puede tener diferentes modos de direccionamiento.

					S	Z	H	P V	N	C
Registro Directo A – r	Cp r donde r es alguno de los siguientes registros de 8 bits: B, C, D, E, H, L, A	1	4	B ... 10111--- donde --- codifica alguno de los registros de 8 bits con la convención de la tabla 1	☒	☒	☒	☒ V	1	☒
Registro Indirecto A - (HL)	Cp (HL)	1	6	BE	☒	☒	☒	☒ V	1	☒
Registro Indexado A - (IX + d)	Cp (IX + d)	3	14	DD BE d	☒	☒	☒	☒ V	1	☒
Registro Indexado A - (IY + d)	Cp (IY + d)	3	14	FD BE d	☒	☒	☒	☒ V	1	☒
Inmediato A – v	Cp v	2	6	FE v	☒	☒	☒	☒ V	1	☒



3.4. Instrucciones de Salto y de Ciclo

JP (Incondicional)

Con esta instrucción se puede alterar la secuencia de ejecución. Existen diversas instrucciones según el modo de direccionamiento del operando.

					S	Z	H	P V	N	C
Inmediato $PC \leftarrow mn$	JP rotulo donde rótulo es un número mn de 16 bits	3	9	C3 n m	☒	☒	☒	☒	☒	☒
Registro Indirecto $PC \leftarrow HL$	JP (HL)	1	3	E9	☒	☒	☒	☒	☒	☒
Registro Indexado $PC \leftarrow IX$	JP (IX)	2	6	DD E9	☒	☒	☒	☒	☒	☒
Registro Indexado $PC \leftarrow IY$	JP (IY)	2	6	FD E9	☒	☒	☒	☒	☒	☒
Relativo $PC \leftarrow PC + (j-2)$	JR rotulo	2	8	18 (j-2)	☒	☒	☒	☒	☒	☒

j: rotulo – dirección de la instrucción



JP (Condicional)

Con esta instrucción se puede cortar la secuencia de ejecución según esté encendido o apagado alguno de los flags del registro F (carry, zero). Existen diversas instrucciones según el modo de direccionamiento del operando.

					S	Z	H	P V	N	C
Inmediato Si f es TRUE PC ← mn sino CONTINUAR	JP f, rotulo donde rótulo es un valor mn de 16 bits y f puede ser alguno de los siguiente valores: NZ, Z, C, NC, PO, PE, P, M (Tabla 6)	3	6 (9)	 n m 11---010 n m donde --- codifica según la tabla 6	☒	☒	☒	☒	☒	☒
Relativo Si C = 1 PC ← PC + (j - 2) sino CONTINUAR	JR C, rotulo	2	6 (8)	38 (j-2)	☒	☒	☒	☒	☒	☒
Relativo si C = 0 PC ← PC + (j - 2) sino CONTINUAR	JR NC, rotulo	2	6 (8)	30 (j-2)	☒	☒	☒	☒	☒	☒
Relativo si Z = 1 PC ← PC + (j-2) sino CONTINUAR	JR Z, rotulo	2	6 (8)	28 (j-2)	☒	☒	☒	☒	☒	☒
Relativo si Z = 0 PC ← PC + (j-2) sino CONTINUAR	JR NZ, rotulo	2	6 (8)	20 (j-2)	☒	☒	☒	☒	☒	☒

j: rotulo – dirección de la instrucción



DJNZ (ciclo)

Con esta instrucción se puede repetir un ciclo la cantidad de veces que se cargue inicialmente en el registro B.

					S	Z	H	P V	N	C
Registro Relativo	DJNZ rotulo	5	7 (9)	10 (j-2)	☒	☒	☒	☒	☒	☒
B ← B-1										
Luego, si B ≠ 0 PC ← PC + (j-2) sino CONTINUAR con próxima instrucción										

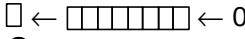
j: rotulo – dirección de la instrucción



3.5. Instrucciones de Manipulación de Bits

Shift (Decalaje)

Permiten reubicar los bits moviéndolos hacia izquierda o derecha.

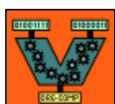
										
					S	Z	H	P V	N	C
Registro Directo para fuente y destino  C <i>shift left arithmetic</i>	SLA r donde r puede ser alguno de los siguientes registros de 8 bits: A,B, C, D, E, H, L	3	7	CB 2... 11001011 00100--- donde --- codifica al registro según la tabla 1	☒	☒	0	☒ P	0	☒
Registro Indirecto para fuente y destino ídem al anterior	SLA (HL)	5	13	CB 26	☒	☒	0	☒ P	0	☒
Registro Indexado para fuente y destino ídem al anterior	SLA (IX+d) donde d es un desplazamiento de 8 bits	7	19	DD CB d 26	☒	☒	0	☒ P	0	☒
Registro Indexado para fuente y destino ídem al anterior	SLA (IY+d) donde d es un desplazamiento de 8 bits	7	19	FD CB d 26	☒	☒	0	☒ P	0	☒



					S	Z	H	P V	N	C
Registro Directo para fuente y destino	SRA r donde r puede ser alguno de los siguientes registros de 8 bits: A, B, C, D, E, H, L	3	7	CB 2... 11001011 00101--- donde --- codifica al registro según la tabla 1	☒	☒	0	☒ P	0	☒
Registro Indirecto para fuente y destino ídem al anterior	SRA (HL)	5	13	CB 2E	☒	☒	0	☒ P	0	☒
Registro Indexado para fuente y destino ídem al anterior	SRA (IX+d) donde d es un desplazamiento de 8 bits	7	19	DD CB d 2E	☒	☒	0	☒ P	0	☒
Registro Indexado para fuente y destino ídem al anterior	SRA (IY+d) donde d es un desplazamiento de 8 bits	7	19	FD CB d 2E	☒	☒	0	☒ P	0	☒



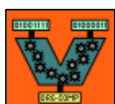
					S	Z	H	P V	N	C
Registro Directo para fuente y destino 0 → → C	SRL r donde r puede ser alguno de los siguientes registros de 8 bits: A, B, C, D, E, H, L	3	7	CB 3... 11001011 00111--- donde --- codifica al registro según la tabla 1	☒	☒	0	☒ P	0	☒
Registro Indirecto para fuente y destino ídem al anterior	SRL (HL)	5	13	CB 3E	☒	☒	0	☒ P	0	☒
Registro Indexado para fuente y destino ídem al anterior	SRL (IX+d) donde d es un desplazamiento de 8 bits	7	19	DD CB d 3E	☒	☒	0	☒ P	0	☒
Registro Indexado para fuente y destino ídem al anterior	SRL (IY+d) donde d es un desplazamiento de 8 bits	7	19	FD CB d 3E	☒	☒	0	☒ P	0	☒



Rotación

Permiten parecido al anterior reubicar los bits moviéndolos dentro del operando, pero realizando un tratamiento circular de los mismos.

					S	Z	H	P V	N	C
Registro Implícito para fuente y destino 	RLA	1	3	17	☒	☒	0	☒ P	0	☒
Registro Directo para fuente y destino ídem al anterior	RL r donde r puede ser alguno de los siguientes registros de 8 bits: A, B, C, D, E, H, L	3	7	CB 1... 11001011 00010--- donde --- codifica al registro según la tabla 1	☒	☒	0	☒ P	0	☒
Registro Indirecto para fuente y destino ídem al anterior	RL (HL)	5	13	CB 16	☒	☒	0	☒ P	0	☒
Registro Indexado para fuente y destino ídem al anterior	RL (IX+d) donde d es un desplazamiento de 8 bits	7	19	DD CB d 16	☒	☒	0	☒ P	0	☒
Registro Indexado para fuente y destino ídem al anterior	RL (IY+d) donde d es un desplazamiento de 8 bits	7	19	FD CB d 16	☒	☒	0	☒ P	0	☒



					S	Z	H	P V	N	C
Registro Implícito para fuente y destino	RLCA	1	3	07	☒	☒	0	☒ P	0	☒
Registro Directo para fuente y destino ídem al anterior	RLC r donde r puede ser alguno de los siguientes registros de 8 bits: A, B, C, D, E, H, L	3	7	CB 0... 11001011 00000--- donde --- codifica al registro según la tabla 1	☒	☒	0	☒ P	0	☒
Registro Indirecto para fuente y destino ídem al anterior	RLC (HL)	5	13	CB 06	☒	☒	0	☒ P	0	☒
Registro Indexado para fuente y destino ídem al anterior	RLC (IX+d) donde d es un desplazamiento de 8 bits	7	19	DD CB d 06	☒	☒	0	☒ P	0	☒
Registro Indexado para fuente y destino ídem al anterior	RLC (IY+d) donde d es un desplazamiento de 8 bits	7	19	FD CB d 06	☒	☒	0	☒ P	0	☒



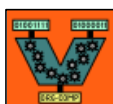
					S	Z	H	P V	N	C
Registro Implícito para fuente y destino	RRA	1	3	1F	☒	☒	0	☒ P	0	☒
Registro Directo Para fuente y destino ídem al anterior	RR r donde r puede ser alguno de los siguientes registros de 8 bits: A, B, C, D, E, H, L	3	7	CB 1... 11001011 00011--- donde --- codifica al registro según la tabla 1	☒	☒	0	☒ P	0	☒
Registro Indirecto para fuente y destino ídem al anterior	RR (HL)	5	13	CB 1E	☒	☒	0	☒ P	0	☒
Registro Indexado para fuente y destino ídem al anterior	RR (IX+d) donde d es un desplazamiento de 8 bits	7	19	DD CB d 1E	☒	☒	0	☒ P	0	☒
Registro Indexado para fuente y destino ídem al anterior	RR (IY+d) donde d es un desplazamiento de 8 bits	7	19	FD CB d 1E	☒	☒	0	☒ P	0	☒



					S	Z	H	P V	N	C
Registro Implícito para fuente y destino	RRCA	1	3	0F	☒	☒	0	☒ P	0	☒
Registro Directo para fuente y destino ídem al anterior	RRC r donde r puede ser alguno de los siguientes registros de 8 bits: A, B, C, D, E, H, L	3	7	CB 0... 11001011 00001--- donde --- codifica al registro según la tabla 1	☒	☒	0	☒ P	0	☒
Registro Indirecto para fuente y destino ídem al anterior	RRC (HL)	5	13	CB 0E	☒	☒	0	☒ P	0	☒
Registro Indexado para fuente y destino ídem al anterior	RRC (IX+d) donde d es un desplazamiento de 8 bits	7	19	DD CB d 0E	☒	☒	0	☒ P	0	☒
Registro Indexado para fuente y destino ídem al anterior	RRC (IY+d) donde d es un desplazamiento de 8 bits	7	19	FD CB d 0E	☒	☒	0	☒ P	0	☒



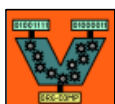
					S	Z	H	P V	N	C
Registro Implícito para fuentes y destinos entre Registro A y (HL) Sean $A = N_3N_2$ y $(HL) = N_1N_0$ esta operación mueve nibbles dejando $A = N_3N_1$ y $(HL) = N_0N_2$	RLD	8	16	ED 6F	☒	☒	0	☒ P	0	☒
Registro Implícito para fuentes y destinos entre Registro A y (HL) Sean $A = N_3N_2$ y $(HL) = N_1N_0$ esta operación mueve nibbles dejando $A = N_3N_0$ y $(HL) = N_2N_1$	RRD	8	16	ED 67	☒	☒	0	☒ P	0	☒



Seteo de Bits

Permite setear o apagar bits dentro del operando.

					S	Z	H	P V	N	C
Registro Directo para r y Dir. a bit para b bit b de r $\leftarrow$ 1	Set b, r donde b es la posición del bit dentro del registro y r puede ser alguno de los siguientes registros de 8 bits: A, B, C, D, E, H, L	3	7	CB ... 11001011 11--- --- donde las primeras --- codifican a b y las segundas al registro según la tabla 1	☒	☒	☒	☒	☒	☒
Registro Indirecto para r y Dir. a bit para b bit b de (HL) $\leftarrow$ 1	Set b,(HL)	5	13	CB ... 11001011 11--- 110 donde --- codifica a b	☒	☒	☒	☒	☒	☒
Registro Indexado para r y Dir. a bit para b bit b de (IX+d) $\leftarrow$ 1	Set b, (IX+d) donde d es un desplazamiento de 8 bits	7	19	DD CB d ... 11011101 11001011 d 11---110 donde --- codifica a b	☒	☒	☒	☒	☒	☒
Registro Indexado para r y Dir. a bit para b bit b de (IY+d) $\leftarrow$ 1	Set b, (IY+d) donde d es un desplazamiento de 8 bits	7	19	FD CB d ... 11111101 11001011 d 11---110 donde --- codifica a b	☒	☒	☒	☒	☒	☒



					S	Z	H	P V	N	C
Registro Directo para r y Dir. a bit para b bit b de r $\leftarrow$ 0	Res b, r donde b es la posición del bit dentro del registro y r puede ser alguno de los siguientes registros de 8 bits: A, B, C, D, E, H, L	3	7	CB 11001011 10--- --- donde las primeras --- codifican a b y las segundas al registro según la tabla 1	☒	☒	☒	☒	☒	☒
Registro Indirecto para r y Dir. a bit para b bit b de (HL) $\leftarrow$ 0	Res b,(HL)	5	13	CB 11001011 10--- 110 donde --- codifica a b	☒	☒	☒	☒	☒	☒
Registro Indexado para r y Dir. a bit para b bit b de (IX+d) $\leftarrow$ 0	Res b, (IX+d) donde d es un desplazamiento de 8 bits	7	19	DD CB d ... 11011101 11001011 d 10---110 donde --- codifica a b	☒	☒	☒	☒	☒	☒
Registro Indexado para r y Dir. a bit para b bit b de (IY+d) $\leftarrow$ 0	Res b, (IY+d) donde d es un desplazamiento de 8 bits	7	19	FD CB d ... 11111101 11001011 d 10---110 donde --- codifica a b	☒	☒	☒	☒	☒	☒



3.6. Instrucciones para el Manejo de Stack

Push

Coloca el contenido del registro de 16 bits especificado en el stack y actualiza el stack pointer.

					S	Z	H	P V	N	C
Registro Directo para el fuente e Implícito para el destino $(SP-1) \leftarrow hi\ r$ $(SP-2) \leftarrow lo\ r$ $SP \leftarrow SP - 2$	PUSH r donde r es alguno de los registros de 16 bits de la tabla 2	5	11	... 5 11--0101 donde -- codifica alguno de los registros de la tabla 2	☒	☒	☒	☒	☒	☒
Registro Implícito para fuente y destino ídem	PUSH IX	6	14	DD E5	☒	☒	☒	☒	☒	☒
Registro Implícito para fuente y destino ídem	PUSH IY	6	14	FD E5	☒	☒	☒	☒	☒	☒



Pop

Saca del stack 16 bits y los deja en el registro de 16 bits especificado, y actualiza el **stack pointer**.

					S	Z	H	P V	N	C
Registro Directo para el fuente e Implícito para el destino hi r $\leftarrow$ (SP+1) lo r $\leftarrow$ (SP) SP $\leftarrow$ SP +2	POP r donde r es alguno de los registros de 16 bits de la tabla 2	1	9	... 11--0001 donde -- codifica alguno de los registros de la tabla 2	☒	☒	☒	☒	☒	☒
Registro Implícito para fuente y destino ídem	POP IX	2	12	DD E1	☒	☒	☒	☒	☒	☒
Registro Implícito para fuente y destino ídem	POP IY	2	12	FD E1	☒	☒	☒	☒	☒	☒



Call

Realiza un salto hacia el rótulo especificado, previo salvar la dirección del registro PC (Program Counter) en el stack.

					S	Z	H	P V	N	C
Inmediato	CALL mn	6	16	CD n m	☒	☒	☒	☒	☒	☒
(SP-1) ← hi PC										
(SP-2) ← lo PC										
PC ← mn										
SP ← SP -2										



Ret

Realiza un salto hacia la dirección que se encuentra en el tope del stack.

					S	Z	H	P V	N	C
Implícito hi PC $\leftarrow$ (SP+1) lo PC $\leftarrow$ (SP) SP $\leftarrow$ SP + 2	RET	3	9	C9	☒	☒	☒	☒	☒	☒



3.7. Instrucciones Especiales de Control

DAA

Esta instrucción ajusta el acumulador de manera tal que se obtenga la representación numérica BCD.

					S	Z	H	P V	N	C
Registro Implícito en A para fuente y destino Ajuste Decimal Aritmético	DAA	2	4	27	☒	☒	☒	☒ P	☒	☒

NOP

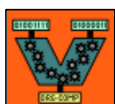
Esta instrucción no tiene efecto alguno sobre memoria y registros, pero consume tiempo. Se la utiliza en rutinas de manejo de tiempo.

					S	Z	H	P V	N	C
Sin operandos No opera	NOP	1	3		☒	☒	☒	☒	☒	☒



4. Cartilla Reducida de Instrucciones (Ordenada por Nombre de Instrucción)

Grupo										
					S	Z	H	P V	N	C
ADC (8 bits)	Adc A, r	2	4	8 ... 10001--- tabla 1	☒	☒	☒	☒ V	0	☒
	Adc A, (HL)	6	6	8E	☒	☒	☒	☒ V	0	☒
	Adc A, (IX + d)	6	14	DD 8E d	☒	☒	☒	☒ V	0	☒
	Adc A, (IY + d)	6	14	FD 8E d	☒	☒	☒	☒ V	0	☒
	Adc A, v	2	6	CE v	☒	☒	☒	☒ V	0	☒
ADC (16 bits)	Adc HL, r	6	10	ED ...A 11101101 01--1010 tabla 3	☒	☒	?	☒ V	0	☒
ADD (8 bits)	Add A, r	22	4	8 ... 10000--- tabla 1	☒	☒	☒	☒ V	0	☒
	Add A, (HL)	2	6	86	☒	☒	☒	☒ V	0	☒
	Add A, (IX + d)	6	3	DD 86 d	☒	☒	☒	☒ V	0	☒
	Add A, (IY + d)	6	3	FD 86 d	☒	☒	☒	☒ V	0	☒
	Add A, v	2	6	C6 v	☒	☒	☒	☒ V	0	☒



Grupo										
					S	Z	H	P V	N	C
ADD (16 bits)	Add HL, r	5	7	... 9 00--1001 tabla 3	☒	☒	?	☒	0	☒
	Add IX, r	6	10	DD ...9 11011101 00--1001 tabla 4	☒	☒	?	☒	0	☒
	Add IY, r	6	10	FD ...9 11111101 00—1001 tabla 5	☒	☒	?	☒	0	☒
AND	And r	1	4	A ... 10100--- tabla 1	☒	☒	1	☒ P	0	0
	And (HL)	1	6	A6	☒	☒	1	☒ P	0	0
	And (IX + d)	3	14	DD A6 d	☒	☒	1	☒ P	0	0
	And (IY + d)	3	14	FD A6 d	☒	☒	1	☒ P	0	0
	And v	2	6	E6 v	☒	☒	1	☒ P	0	0
CALL	CALL mn	6	16	CD n m	☒	☒	☒	☒	☒	☒
CP	Cp r	1	4	B ... 10111--- tabla 1	☒	☒	☒	☒ V	1	☒
	Cp (HL)	1	6	BE	☒	☒	☒	☒ V	1	☒
	Cp (IX + d)	3	14	DD BE d	☒	☒	☒	☒ V	1	☒
	Cp (IY + d)	3	14	FD BE d	☒	☒	☒	☒ V	1	☒
	Cp v	2	6	FE v	☒	☒	☒	☒ V	1	☒



Grupo										
					S	Z	H	P V	N	C
CPL	Cpl	1	3	2F	☒	☒	1	☒	1	☒
DAA	Daa	2	4	27	☒	☒	☒	☒ P	☒	☒
DEC (8 bits)	Dec r	2	4	 00---101 tabla 1	☒	☒	☒	☒ V	1	☒
	Dec (HL)	4	10	35	☒	☒	☒	☒ V	1	☒
	Dec (IX + d)	8	18	DD 35 d	☒	☒	☒	☒ V	1	☒
	Dec (IY + d)	8	18	FD 35 d	☒	☒	☒	☒ V	1	☒
DEC (16 bits)	Dec r	2	4	... B 00--1011 tabla 3	☒	☒	☒	☒	☒	☒
	Dec IX	3	7	DD 2B	☒	☒	☒	☒	☒	☒
	Dec IY	3	7	FD 2B	☒	☒	☒	☒	☒	☒
DJNZ	Djnz rotulo	5	7 (9)	10 (j-2)	☒	☒	☒	☒	☒	☒
INC (8 bits)	Inc r	2	4	 00---100 tabla 1	☒	☒	☒	☒ V	0	☒
	Inc (HL)	4	10	34	☒	☒	☒	☒ V	0	☒
	Inc (IX + d)	8	18	DD 34 d	☒	☒	☒	☒ V	0	☒
	Inc (IY + d)	8	18	FD 34 d	☒	☒	☒	☒ V	0	☒



Grupo										
					S	Z	H	P V	N	C
INC (16 bits)	Inc r	2	4	... 3 00--0011 tabla 3	☒	☒	☒	☒	☒	☒
	Inc IX	3	7	DD 23	☒	☒	☒	☒	☒	☒
	Inc IY	3	7	FD 23	☒	☒	☒	☒	☒	☒
JP	JP rotulo	3	9	C3 n m	☒	☒	☒	☒	☒	☒
	JP (HL)	1	3	E9	☒	☒	☒	☒	☒	☒
	JP (IX)	2	6	DD E9	☒	☒	☒	☒	☒	☒
	JP (IY)	2	6	FD E9	☒	☒	☒	☒	☒	☒
	JP f, rotulo f es valor de tabla 6	3	6 (9)	 n m 11---010 n m tabla 6	☒	☒	☒	☒	☒	☒
JR	JR rotulo	2	8	18 (j-2)	☒	☒	☒	☒	☒	☒
	JR C, rotulo	2	6 (8)	38 (j-2)	☒	☒	☒	☒	☒	☒
	JR NC, rotulo	2	6 (8)	30 (j-2)	☒	☒	☒	☒	☒	☒
	JR Z, rotulo	2	6 (8)	28 (j-2)	☒	☒	☒	☒	☒	☒
	JR NZ, rotulo	2	6 (8)	20 (j-2)	☒	☒	☒	☒	☒	☒



Grupo										
					S	Z	H	P V	N	C
LD (8 bits)	Ld A, I	2	6	ED 57	☒	☒	0	☒	0	☒
	Ld A, R	2	6	ED 5F	☒	☒	0	☒	0	☒
	Ld I, A	2	6	ED 47	☒	☒	☒	☒	☒	☒
	Ld R, A	2	6	ED 4F	☒	☒	☒	☒	☒	☒
	Ld A, (BC)	2	6	0A	☒	☒	☒	☒	☒	☒
	Ld A, (DE)	2	6	1A	☒	☒	☒	☒	☒	☒
	Ld (BC), A	6	7	02	☒	☒	☒	☒	☒	☒
	Ld (DE), A	3	7	12	☒	☒	☒	☒	☒	☒
	Ld r, r'	2	4	 01--- --- r en tabla 1 r' en tabla 1	☒	☒	☒	☒	☒	☒
	Ld A, (mn)	4	12	3A n m	☒	☒	☒	☒	☒	☒
	Ld (mn), A	5	13	32 n m	☒	☒	☒	☒	☒	☒
	Ld r, (HL)	2	6	 01---110 tabla 1	☒	☒	☒	☒	☒	☒
continúa ↓	Ld (HL), r	3	7	7 ... 01110--- tabla 1	☒	☒	☒	☒	☒	☒



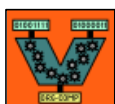
Grupo										
					S	Z	H	P V	N	C
LD (8 bits)	Ld r, (IX+d)	6	14	DD ... d 11011101 01---110 d tabla 1	☒	☒	☒	☒	☒	☒
	Ld r, (IY+d)	6	14	FD ... d 11111101 01---110 d la tabla 1	☒	☒	☒	☒	☒	☒
	Ld (IX+d), r	7	15	DD 7... d 11011101 01110--- d la tabla 1	☒	☒	☒	☒	☒	☒
	Ld (IY+d), r	7	15	FD 7... d 11111101 01110--- d tabla 1	☒	☒	☒	☒	☒	☒
	Ld r, v	2	6	 v 00---110 v tabla 1	☒	☒	☒	☒	☒	☒
	Ld (HL), v	3	9	36 v	☒	☒	☒	☒	☒	☒
	Ld (IX+d), v	5	15	DD 36 d v	☒	☒	☒	☒	☒	☒
	Ld (IY+d), v	5	15	FD 36 d v	☒	☒	☒	☒	☒	☒



Grupo										
					S	Z	H	P V	N	C
LD (16 bits)	Ld SP, HL	2	4	F9	☒	☒	☒	☒	☒	☒
	Ld SP, IX	3	7	DD F9	☒	☒	☒	☒	☒	☒
	Ld SP, IY	3	7	FD F9	☒	☒	☒	☒	☒	☒
	Ld HL, (mn)	3	15	2A n m	☒	☒	☒	☒	☒	☒
	Ld IX, (mn)	6	18	DD 2A n m	☒	☒	☒	☒	☒	☒
	Ld IY, (mn)	6	18	FD 2A n m	☒	☒	☒	☒	☒	☒
	Ld (mn), HL	6	16	22 n m	☒	☒	☒	☒	☒	☒
	Ld (mn), IX	7	19	DD 22 n m	☒	☒	☒	☒	☒	☒
	Ld (mn), IY	7	19	FD 22 n m	☒	☒	☒	☒	☒	☒
	Ld r, (mn)	6	9	ED...B n m 11101101 01--1011 n m	☒	☒	☒	☒	☒	☒
	Ld (mn), r	7	19	ED...3 n m 11101101 01--0011 n m	☒	☒	☒	☒	☒	☒
	Ld IX, mn	4	12	DD 21 n m	☒	☒	☒	☒	☒	☒
	Ld IY, mn	4	12	FD 21 n m	☒	☒	☒	☒	☒	☒
	Ld r, mn	3	9	...1 n m 00--0001 n m tabla 3	☒	☒	☒	☒	☒	☒



Grupo										
					S	Z	H	P V	N	C
MLT	Mlt r	13	17	ED ...C 11101101 01--1100 tabla 3	☒	☒	☒	☒	☒	☒
NEG	Neg	2	6	ED 44	☒	☒	☒	☒ V	1	☒
NOP	NOP	1	3	00	☒	☒	☒	☒	☒	☒
OR	Or r	1	4	B ... 10110--- tabla 1	☒	☒	0	☒ P	0	0
	Or (HL)	1	6	B6	☒	☒	0	☒ P	0	0
	Or (IX + d)	3	14	DD B6 d	☒	☒	0	☒ P	0	0
	Or (IY + d)	3	14	FD B6 d	☒	☒	0	☒ P	0	0
	Or v	2	6	F6 v	☒	☒	0	☒ P	0	0
POP	POP r	1	9	... 11--0001 tabla 2	☒	☒	☒	☒	☒	☒
	POP IX	2	12	DD E1	☒	☒	☒	☒	☒	☒
	POP IY	2	12	FD E1	☒	☒	☒	☒	☒	☒
PUSH	PUSH r	5	11	... 5 11--0101 tabla 2	☒	☒	☒	☒	☒	☒
	PUSH IX	6	14	DD E5	☒	☒	☒	☒	☒	☒
	PUSH IY	6	14	FD E5	☒	☒	☒	☒	☒	☒



Grupo										
					S	Z	H	P V	N	C
RES	Res b, (HL) b es lposición del bit	5	13	CB 11001011 10--- 110	☒	☒	☒	☒	☒	☒
	Res b, (IX+d) b es posición del bit	7	19	DD CB d ... 11011101 11001011 d 10---110	☒	☒	☒	☒	☒	☒
	Res b, (IY+d) b es posición del bit	7	19	FD CB d ... 11111101 11001011 d 10---110	☒	☒	☒	☒	☒	☒
	Res b, r b es posición del bit	3	7	CB 11001011 10--- --- primero b segundo tabla 1	☒	☒	☒	☒	☒	☒
RET	RET	3	9	C9	☒	☒	☒	☒	☒	☒
Rotation continúa ↓	RLA	1	3	17	☒	☒	0	☒ P	0	☒
	RL r	3	7	CB 1... 11001011 00010--- tabla 1	☒	☒	0	☒ P	0	☒
	RL (HL)	5	13	CB 16	☒	☒	0	☒ P	0	☒
	RL (IX+d)	7	19	DD CB d 16	☒	☒	0	☒ P	0	☒
	RL (IY+d)	7	19	FD CB d 16	☒	☒	0	☒ P	0	☒
	RLCA	1	3	07	☒	☒	0	☒ P	0	☒
	RLC (HL)	5	13	CB 06	☒	☒	0	☒ P	0	☒



Grupo										
					S	Z	H	P V	N	C
Rotation	RLC r	3	7	CB 0... 11001011 00000--- tabla 1	☒	☒	0	☒ P	0	☒
	RLC (IX+d)	7	19	DD CB d 06	☒	☒	0	☒ P	0	☒
	RLC (IY+d)	7	19	FD CB d 06	☒	☒	0	☒ P	0	☒
	RLD	8	16	ED 6F	☒	☒	0	☒ P	0	☒
	RRA	1	3	1F	☒	☒	0	☒ P	0	☒
	RR r	3	7	CB 1... 11001011 00011--- tabla 1	☒	☒	0	☒ P	0	☒
	RR (HL)	5	13	CB 1E	☒	☒	0	☒ P	0	☒
	RR (IX+d)	7	19	DD CB d 1E	☒	☒	0	☒ P	0	☒
	RR (IY+d)	7	19	FD CB d 1E	☒	☒	0	☒ P	0	☒
	RRCA	1	3	0F	☒	☒	0	☒ P	0	☒
	RRC r	3	7	CB 0... 11001011 00001--- tabla 1	☒	☒	0	☒ P	0	☒
	RRC (HL)	5	13	CB 0E	☒	☒	0	☒ P	0	☒
	RRC (IX+d)	7	19	DD CB d 0E	☒	☒	0	☒ P	0	☒
	RRC (IY+d)	7	19	FD CB d 0E	☒	☒	0	☒ P	0	☒
	RRD	8	16	ED 67	☒	☒	0	☒ P	0	☒



Grupo										
					S	Z	H	P V	N	C
SET	Set b, r b es posición del bit	3	7	CB ... 11001011 11--- --- tabla 1	☒	☒	☒	☒	☒	☒
	Set b,(HL) b es posición del bit	5	13	CB ... 11001011 11--- 110	☒	☒	☒	☒	☒	☒
	Set b, (IX+d) b es posición del bit	7	19	DD CB d ... 11011101 11001011 d 11---110	☒	☒	☒	☒	☒	☒
	Set b, (IY+d) b es posición del bit	7	19	FD CB d ... 11111101 11001011 d 11---110	☒	☒	☒	☒	☒	☒
SBC (8 bits)	Sbc A, r	2	4	9 ... 10011--- tabla 1	☒	☒	☒	☒ V	1	☒
	Sbc A, (HL)	2	6	9E	☒	☒	☒	☒ V	1	☒
	Sbc A, (IX + d)	6	14	DD 9E d	☒	☒	☒	☒ V	1	☒
	Sbc A, (IY + d)	6	14	FD 9E d	☒	☒	☒	☒ V	1	☒
	Sbc A, v	2	6	DE v	☒	☒	☒	☒ V	1	☒
SBC (16 bits)	Sbc HL, r	6	10	ED ...2 11101101 01--0010 tabla 3	☒	☒	?	☒ V	1	☒



Grupo										
					S	Z	H	P V	N	C
Shift	SLA r	3	7	CB 2... 11001011 00100--- tabla 1	☒	☒	0	☒ P	0	☒
	SLA (HL)	5	13	CB 26	☒	☒	0	☒ P	0	☒
	SLA (IX+d)	7	19	DD CB d 26	☒	☒	0	☒ P	0	☒
	SLA (IY+d)	7	19	FD CB d 26	☒	☒	0	☒ P	0	☒
	SRA r	3	7	CB 2... 11001011 00101--- tabla 1	☒	☒	0	☒ P	0	☒
	SRA (HL)	5	13	CB 2E	☒	☒	0	☒ P	0	☒
	SRA (IX+d)	7	19	DD CB d 2E	☒	☒	0	☒ P	0	☒
	SRA (IY+d)	7	19	FD CB d 2E	☒	☒	0	☒ P	0	☒
	SRL r	3	7	CB 3... 11001011 00111--- tabla 1	☒	☒	0	☒ P	0	☒
	SRL (HL)	5	13	CB 3E	☒	☒	0	☒ P	0	☒
	SRL (IX+d)	7	19	DD CB d 3E	☒	☒	0	☒ P	0	☒
	SRL (IY+d)	7	19	FD CB d 3E	☒	☒	0	☒ P	0	☒



Grupo										
					S	Z	H	P V	N	C
SUB (8 bits)	Sub r	2	4	9 ... 10010--- tabla 1	☒	☒	☒	☒ V	1	☒
	Sub (HL)	2	6	96	☒	☒	☒	☒ V	1	☒
	Sub (IX + d)	6	14	DD 96 d	☒	☒	☒	☒ V	1	☒
	Sub (IY + d)	6	14	FD 96 d	☒	☒	☒	☒ V	1	☒
	Sub v	2	6	D6 v	☒	☒	☒	☒ V	1	☒
XOR	XOR r	1	4	A ... 10101--- tabla 1	☒	☒	0	☒ P	0	0
	XOR (HL)	1	6	AE	☒	☒	0	☒ P	0	0
	XOR (IX + d)	3	14	DD AE d	☒	☒	0	☒ P	0	0
	XOR (IY + d)	3	14	FD AE d	☒	☒	0	☒ P	0	0
	XOR v	2	6	EE v	☒	☒	0	☒ P	0	0



5. Simulador del Procesador Z-80

Cada computador tiene un procesador de una marca y modelo definido, es decir que cuenta con un set de instrucciones que no necesariamente coinciden con las de un procesador de otra marca y/o modelo. Al haber distintos set de instrucciones, un programa assembler escrito para un procesador definido no sirve para ser ejecutado en otro. Por este motivo existen distintas variantes del lenguaje Assembler para cada uno de los distintos procesadores disponibles.

Si se trabaja con computadoras hogareñas que cuentan con procesadores de la familia Intel 80x86, no es posible ejecutar sobre las mismas programas en Assembler Z-80. Para poder utilizar ese assembler necesitamos trabajar con un Simulador de Z-80.

Un simulador de un procesador, en general, es un programa que se ejecuta en un procesador obviamente diferente al que simula y permite, valga la redundancia, simular la ejecución de programas escritos para el procesador simulado. El simulador genera un procesador “virtual” que entiende un lenguaje de máquina específico y permite ejecutar instrucciones, analizando como afecta la ejecución de las mismas a cada uno de los elementos del procesador.

Para simular la ejecución de un programa escrito en lenguaje Assembler Z-80180, se deben seguir los siguientes pasos:

- ☞ Escritura del programa fuente.
- ☞ Ensamblado del programa fuente.
- ☞ Linkedición del código objeto (eventualmente con otros módulos).
- ☞ Carga del programa en el simulador.

5.1. Escritura del Programa Fuente

El primer paso consiste en escribir el programa en lenguaje Assembler Z80180 en cualquier editor de texto (Edit de DOS, Notepad de Windows, etc). El nombre del archivo fuente debe tener la extensión **.asm**. (Ejemplo: prog1.asm).

Para que el programa sea válido, es decir, que pueda ser entendido por el ensamblador, deben seguirse estrictamente las siguientes reglas:

- Cada línea del texto debe contener una y solo una instrucción.
- Se deben usar nombres mnemotécnicos para las instrucciones.
- Cada línea del programa debe tener una estructura legal (ver cartilla de instrucciones).
- Colocar para indicar la finalización del programa la pseudo-instrucción **end**.
- Se debe respetar el encolumnados de rótulos e instrucciones (los rótulos deben empezar en la primera columna).
- Verificar que en el archivo fuente no queden líneas en blanco al final del archivo.
- Agregar comentarios que aclaren la lógica del programa.



Ejemplo: archivo **pgm1.asm**

```

      aseg
start  org      2000h      ;el prog se carga en 2000h
      ld        A, 10      ;carga en el reg A el valor 10
      ld        valor, A   ;grabo en "valor" el contenido de A
      rst       38h        ;fin
valor: db        0         ;reservo espacio para almacenar
      end       start      ;indico donde empieza el programa
    
```

5.2. Ensamblado del Programa Fuente

El ensamblador es un programa que se ejecuta en D.O.S., cuyo nombre es “**zas**”. Su función es chequear la correctitud del código, detectando errores de sintaxis, rótulos locales no definidos, instrucciones inexistentes, etc. En caso de no detectar errores, el ensamblador convierte el código fuente en código objeto.

Para ensamblar un código se debe ejecutar en D.O.S. la siguiente línea de comando:

```
c:> zas [-lNombreListado] [-wAncho] ArchivoFuente
```

La opción **-l** se especifica si se desea que el ensamblador genere un archivo (NombreListado.**lst**) con un listado, conteniendo:

- Código fuente y su traducción a código de máquina.
- Valor del Contador de Posiciones.
- Si se produjeron errores, una letra en cada línea con errores señalando el tipo de error.
- Tabla de símbolos interna del programa.

La opción **-w** especifica el ancho (en cantidad de columnas) que tendrá el listado.

Por último, el único argumento obligatorio es “**ArchivoFuente**” que es el nombre del archivo conteniendo el código fuente.

Si se produjeran errores durante el ensamblado, el listado de los mismos se realizará por pantalla, indicando el tipo de error y la línea en que se produjo.

La línea de comando para ensamblar el código fuente escrito en el ejemplo es la siguiente:

```
c:> zas -lpgm1 -w75 pgm1.asm
```



y la respuesta del ensamblador por pantalla es la siguiente:

```
Z-World Z80/HD64180 Macro Assembler Version 1.08
pgm1.asm:4:      Operand error
abs = 8196 bytes
```

La segunda línea de este mensaje nos indica que el ensamblador no pudo finalizar su traducción con éxito debido a un error en el código fuente. Dicho error se encuentra en la línea 4 y es un error en los operandos. Para verlo con más claridad, podemos editar el archivo **pgm1.lst** (generado por el proceso de ensamblado) cuyo contenido es el siguiente:

```
1      0000      aseg
2      2000      org 2000h ;el prog se carga en 2000h
3      2000 3E 0A start ld A, 10 ;carga en el reg A el valor 10
40     2002 FF      rst 38h ;grabo en valor el contenido de A
5      2003 00      valor: db 0 ;fin
6      2004      end start ;reservo espacio para almacenar
7                        ;indico donde empieza el programa
```

```
Z-World Engineering z80 Macro Assembler:
Sat Dec 12 20:29:33 1999
```

```
Page 1
```

```
----- Symbol Table -----
```

```
(ABS) 2004# code 0000# start 2000 valor 2003
abs = 8196 bytes
```

Como se observa en este archivo, el código fuente contenía un error en la instrucción resaltada. El ensamblador lo indica con una letra "O" al comienzo de la línea que significa que dicha instrucción tiene un error de operandos. Efectivamente, analizando el código se observa que la instrucción correcta sería **ld (valor), A**, agregando paréntesis alrededor del rótulo **valor**.

El paso siguiente es editar el archivo **pgm1.asm** nuevamente y corregir la instrucción errónea. Una vez corregida, se vuelve a ensamblar repitiendo en comando anterior. De no detectarse nuevos mensajes de error, el ensamblado se considera exitoso obteniendo el archivo **pgm1.obj**.

5.3. Linkedición del Código Objeto

El linkeditor es un programa que se encarga de generar un único programa ejecutable a partir de uno o más códigos objeto. Resuelve las referencias externas y ordena los segmentos de datos y de código.

Para invocar al linkeditor (el programa **link**, ejecutable en D.O.S) se utiliza la siguiente línea de comando:

```
c:> link -oNombreSalida.cpm -s -m -pcode=DirCod,data=DirDat ListaObjetos
```



La opción **-o** especifica el nombre que se le dará al archivo de salida, resultado del proceso de linkedición.

La opción **-s** especifica que se quiere generar la tabla de símbolos en un archivo cuyo nombre será **NombreSalida.sym**.

La opción **-m** se utiliza para generar un mapa de memoria en un archivo cuyo nombre será **NombreSalida.map**. Este archivo contiene una lista con cada código fuente o módulo linkeado y la dirección de comienzo en que cada uno está alocado; una tabla de símbolos externos ordenados en forma alfabética.

La opción **-pcode=** permite indicar en que dirección se alojará el segmento de código y **data=** para indicar en que dirección se alojará el segmento de datos. Esta opción es inválida si el código es absoluto (se empleó en el código fuente la directiva al compilador **Aseg**). La dirección debe estar en hexadecimal (con el indicador *h* al final). Si no se especificara **data=** el segmento de datos se alojará a continuación del segmento de código.

Por último **ListaDeObjetos** es la lista de nombres de los archivos que contienen los códigos objeto que conformarán el ejecutable, separados por un espacio.

Ejemplos de invocación al linkeditor con más de un código objeto:

```
link -oSalida.cpm -s -pcode=2000h,data=2100h prog1.obj prog2.obj
link -oSalida.cpm -s -pcode=2000h prog.obj prog2.obj
link -oSalida.cpm -s prog1.obj prog2.obj
link -oSalida.cpm -m prog1.obj prog2.obj
```

Para linkeditar nuestro ejemplo la línea de comando es la siguiente:

```
c:> link -opgm1.cpm pgm1.obj
```

5.4. Carga del Programa Convertido en el Simulador

Finalmente se debe cargar en el simulador el archivo **.cpm** obtenido en la linkedición.

Todos los programas ejecutados en los pasos anteriores tomaban un archivo fuente y producían uno o más archivos de resultado. En este caso, no se produce una salida sino que se ingresa a la pantalla de simulación.

Desde esta pantalla se puede ejecutar un programa paso a paso y observar los efectos que produce sobre los flags, los registros y la memoria, la ejecución de una o varias instrucciones.



6. Tabla de Códigos ASCII

Valor ASCII	Caracter
20	Space
21	!
22	"
23	#
24	\$
25	%
26	&
27	'
28	(
29	)
2A	*
2B	+
2C	,
2D	-
2E	.
2F	/
30	0
31	1
32	2
33	3
34	4
35	5
36	6
37	7
38	8
39	9
3A	:
3B	;
3C	<
3D	=
3E	>
3F	?

Valor ASCII	Caracter
40	@
41	A
42	B
43	C
44	D
45	E
46	F
47	G
48	H
49	I
4A	J
4B	K
4C	L
4D	M
4E	N
4F	O
50	P
51	Q
52	R
53	S
54	T
55	U
56	V
57	W
58	X
59	Y
5A	Z
5B	[
5C	\
5D	]
5E	^
5F	_

Valor ASCII	Caracter
60	`
61	a
62	b
63	c
64	d
65	e
66	f
67	g
68	h
69	i
6A	j
6B	k
6C	l
6D	m
6E	n
6F	o
70	p
71	q
72	r
73	s
74	t
75	u
76	v
77	w
78	x
79	y
7A	z
7B	{
7C	
7D	}
7E	~
7F	DEL